

Carbon Clean: System Architecture Document

Anjan Mahapatra

May 23, 2026

Abstract

This document presents the comprehensive system architecture for Carbon Clean, the leading full-stack industrial carbon capture platform focused on B2B decarbonization, Capture-as-a-Service (CaaS), carbon credit monetization, and IoT-enabled performance monitoring. The architecture is designed to support 1,200+ industrial partners, 50M+ tonnes of annual CO capture capacity, 100,000+ IoT sensors, \$4.5B in facilitated project financing AUM, and \$1.28B+ in annual revenue by FY30F.

The architecture follows a microservices-based, cloud-native design with IoT-first data ingestion from capture units, real-time carbon credit issuance, and AI-driven performance optimization. Key components include: (1) multi-channel industrial partner onboarding system, (2) Carbon Clean Credit Engine for project underwriting (82% default prediction accuracy), (3) CaaS (Capture-as-a-Service) billing and collections module, (4) IoT-based capture unit monitoring platform with real-time efficiency alerting, (5) Carbon credit issuance and monetization engine, (6) CycloneCC remote diagnostics and predictive maintenance, (7) B2B partner portal for real-time capture performance, (8) logistics and CO transport optimization engine, (9) carbon price hedging and risk management platform, and (10) analytics and ESG reporting layer.

The architecture emphasizes scalability (horizontal scaling to 10M+ daily API requests), fault tolerance (99.95% uptime target, 2-hour RTO), security (AES-256 encryption, RBAC, mTLS for IoT devices), and compliance with Verra/Gold Standard MRV requirements, EU ETS, US 45Q, and emerging carbon market regulations.

Contents

1	Architecture Overview	4
1.1	Design Principles	4
1.2	User Personas Supported	4
1.3	High-Level Architecture Layers	6
2	Client Layer	7
2.1	Partner Portal (Web)	7
2.2	Field Service Mobile App (iOS/Android)	7
2.3	Corporate API Access	8
2.4	Carbon Credit Registry Portal	8
3	API Gateway Layer	9
3.1	API Versioning Strategy	9
4	Microservices Layer	9
4.1	Microservices Summary	10
4.2	Service 1: Capture Monitoring Service	10
4.3	Service 2: Carbon Credit Engine	11
4.4	Service 3: Credit Underwriting Engine (Carbon Clean Credit Engine)	12
4.5	Service 4: CaaS Billing Service	12
4.6	Service 5: Predictive Maintenance Service	13
4.7	Service 6: Outgrow DAC Management Service	14
4.8	Service 7: Hedging Platform	14
4.9	Service 8: Transport Logistics Service	15
4.10	Service 9: Data Monetization Service	15
4.11	Service 10: Analytics & Reporting Service	16
4.12	Service 11: Notification Service (Cross-cutting)	16
5	Data Layer	17
5.1	Operational Databases	17
5.2	Data Warehouse	17
5.3	Data Lake	18
5.4	Data Pipeline Architecture	18
5.5	Data Governance & Security	19
6	External Integrations	19
6.1	Carbon Registries	19
6.2	Financial Exchanges & Payment Gateways	20
6.3	External APIs	20

7	Scalability & Performance	21
7.1	Scaling Strategy	21
7.2	Auto-scaling Configuration	21
7.3	Performance SLAs	22
8	Deployment Architecture	23
8.1	AWS Infrastructure	23
8.2	Multi-Region Deployment	24
8.3	Disaster Recovery	24
8.4	CI/CD Pipeline	25
9	Offline-First Design	25
9.1	Offline Capabilities by Component	26
9.2	Local Storage Architecture	26
9.3	Sync Conflict Resolution	27
10	Security Architecture	28
10.1	Authentication & Authorization	28
10.2	RBAC Roles and Permissions	29
10.3	Security Controls Summary	30
10.4	IoT Device Security (Capture Unit Sensors)	30
10.5	Compliance Mapping	31
11	Monitoring & Observability	32
11.1	Monitoring Stack	32
11.2	Key Metrics to Monitor	32
11.2.1	Business KPIs (Dashboard - 5 min refresh)	32
11.2.2	Technical KPIs (Dashboard - real-time)	33
11.3	Alerting Rules	33
12	Cost Optimization	34
12.1	Current & Projected Cloud Spend	34
12.2	Optimization Strategies	34
13	Conclusion	35
13.1	Key Architectural Decisions Summary	36
13.2	Projected Capacity by FY30F	36
13.3	Architecture Readiness Assessment	37

1 Architecture Overview

Carbon Clean’s system architecture is built on a microservices-based, cloud-native design deployed on AWS with edge processing for real-time capture unit control. The architecture is designed to handle the unique challenges of industrial carbon capture: real-time telemetry from thousands of sensors, low-latency credit issuance, regulatory compliance across multiple jurisdictions, and integration with carbon registries and financial exchanges.

1.1 Design Principles

The architecture follows seven core design principles:

1. **Microservices-first:** Loosely coupled services that can be developed, deployed, and scaled independently. This allows Carbon Clean to iterate quickly on the credit engine without affecting capture monitoring, and vice versa.
2. **Cloud-native:** Leveraging AWS managed services (EKS, RDS, S3, MSK, IoT Core) for reduced operational overhead. This reduces the need for in-house infrastructure management and allows the engineering team to focus on domain-specific challenges.
3. **IoT-first:** Real-time ingestion and processing of telemetry from 100,000+ capture unit sensors. Every CycloneCC unit generates 10,000+ data points daily—temperature, pressure, flow rate, solvent concentration, CO purity. This data is the foundation of Carbon Clean’s credit underwriting and predictive maintenance.
4. **Real-time carbon credit issuance:** Automated verification and issuance via registry APIs. Traditional credit issuance takes 6+ months; Carbon Clean reduces this to 48 hours, improving partner cash flow and credit pricing.
5. **AI-first:** ML models for capture efficiency optimization, solvent degradation prediction, and carbon price forecasting. These models improve over time as more data is collected, creating a defensible data moat.
6. **Security-by-design:** Encryption at rest and in transit, RBAC, comprehensive audit logging for carbon credits. Carbon credits are financial instruments; their issuance and retirement must be tamper-proof and auditable.
7. **API-first:** Well-documented RESTful APIs for all external integrations. This enables partners to integrate Carbon Clean data into their own ESG reporting systems and allows carbon credit buyers to programmatically purchase credits.

1.2 User Personas Supported

The architecture supports seven primary user personas, each with distinct capabilities and access patterns:

Table 1: User Personas and Capabilities

Persona		Type	Key Capabilities
Industrial ners	Part-	Cement, Steel, Refining	Real-time capture, CaaS billing, credit reports, maintenance
Credit Buyers		Microsoft, Stripe, Airbus	Purchase credits, blockchain audit, retirement certs
Carbon Ops	Clean	Internal	Fleet monitoring, alerts, solvent replacement, dispatch
Field Engineers		Internal	Alerts, diagnostics, service reports, parts ordering
Credit writers	Under-	Internal	Loan assessment, GNPA monitoring, loan structuring
Data API Users		Corporate	Aggregated data, market insights, ESG reports
Registry Auditors		Verra, Gold Standard	Capture verification, credit audit, MRV validation

1.3 High-Level Architecture Layers

The architecture comprises six layers, each with specific responsibilities:

Table 2: Architecture Layers

Layer	Components
Client Layer	Partner portal (React.js web app), field service app (React Native iOS/Android), corporate API access (REST + GraphQL), registry auditor portal (read-only)
API Gateway Layer	Kong / AWS API Gateway for routing, rate limiting (100-50k req/min), authentication (JWT, API keys, mTLS), observability (request logging, tracing)
Microservices Layer	12+ services including Capture Monitoring, Credit Engine, Carbon Credit Engine, CaaS Billing, Predictive Maintenance, Hedging Platform, Transport Logistics, Data Monetization
Data Layer	PostgreSQL (operational - partners, loans, credits), TimescaleDB (time-series IoT telemetry), Redis (caching - sessions, credit prices), Snowflake (warehouse - analytics), S3 (data lake - raw telemetry, KYC), Hyperledger (blockchain - credit audit trail)
External Integrations	Carbon registries (Verra, Gold Standard, CCTS), payment gateways (Stripe, Coinbase), EU ETS futures exchanges (ICE, EEX), reinsurers (Swiss Re, Munich Re), weather APIs (solar forecasting), map APIs (logistics)
Deployment & Observability	EKS on AWS (multi-region: US-East, EU-West-2, ap-south-1), CI/CD (GitHub Actions + ArgoCD), monitoring (Datadog, Prometheus, ELK, PagerDuty)

2 Client Layer

2.1 Partner Portal (Web)

Technology: React.js with TypeScript, Tailwind CSS, real-time WebSocket (Socket.io) for live capture data streaming.

Features:

- **Real-time capture dashboard:** Tonnes captured today, month-to-date (MTD), year-to-date (YTD); capture efficiency (%) vs. target; energy consumption (GJ/tonne) with benchmark comparison; solvent degradation trend (30-day forecast)
- **Carbon credit wallet:** Credits issued (by vintage), credits sold (by buyer), credits retired (by retirement certificate); real-time pricing from AirCarbon/Xpansiv; portfolio performance (realized vs. unrealized gains)
- **CaaS billing:** Monthly invoices (PDF download), payment history (ACH/SEPA/UPI), cost per tonne breakdown (CaaS fee + credit share + financing cost)
- **Maintenance scheduler:** Predictive alerts (solvent replacement recommended in 14 days), service history (date, engineer, work performed), spare parts ordering (with inventory visibility)
- **Compliance reporting:** Automated Verra/Gold Standard reports (MRV templates), EU ETS submissions (annual emissions report), 45Q attestations (IRS Form 8849)
- **ESG dashboard:** Emission reduction certificates (tonnes avoided vs. baseline), sustainability metrics (SDG 13 contribution), audit trail (full transaction history for auditors)

2.2 Field Service Mobile App (iOS/Android)

Technology: React Native + TypeScript + offline-first SQLite (WatermelonDB) for local persistence.

Features:

- **Service ticket management:** Receive real-time alerts (push notification) when predictive maintenance triggers, view unit diagnostics (error codes, sensor readings), check-in/out at site with geotagged timestamps
- **Offline diagnostics:** Capture unit health assessment without internet connectivity (store readings locally, sync when online). Includes solvent sampling (concentration, pH, viscosity), pump vibration analysis, temperature validation.

- **Parts inventory:** Check local stock at warehouse, order replacement parts (with ETA tracking), receive restock alerts for frequently used items (gaskets, seals, sensors)
- **Photo/documentation:** Capture service evidence (before/after photos), upload when connectivity restored, attach to service report for partner visibility
- **Solvent sampling:** Log concentration (%), pH level, viscosity (cP); receive automated recommendations for replacement or adjustment based on ML model predictions

2.3 Corporate API Access

Technology: RESTful APIs (OpenAPI 3.0 specification) + GraphQL (for complex portfolio queries) + API keys + IP whitelisting.

Authentication: API keys (rotated every 90 days) with scope-limited permissions. Corporate buyers can generate multiple keys for different teams (trading desk, ESG reporting, risk management).

Use cases:

- **Carbon credit buyers:** Programmatic credit purchase (POST /v2/credits/purchase), retirement management (POST /v2/credits/retire), portfolio performance (GET /v2/portfolio/holdings, /v2/portfolio/pnl)
- **ESG data aggregators:** Automated capture performance data ingestion (GET /v2/capture/benchmarks, /v2/capture/trends) for inclusion in broader ESG datasets (Bloomberg, MSCI, Sustainalytics)
- **Research institutions:** Anonymized market data access (supply-demand forecasts, price trends, efficiency benchmarks) under academic license (free, with attribution)
- **Government regulators:** Compliance reporting (GET /v2/compliance/verra, /v2/compliance/audit trail access (GET /v2/audit/transactions) with read-only permissions

2.4 Carbon Credit Registry Portal

Integration: Direct API links (REST + webhooks) with Verra (Verification Standard), Gold Standard, and India CCTS.

Features:

- **Automated credit issuance:** Every tonne captured → verified against IoT telemetry (3rd-party oracle) → credit issued via registry API within 48 hours (industry standard is 6+ months)
- **Audit trail:** Blockchain-verified MRV data (timestamped, signed by capture unit) available for registry auditors via read-only API

- **Retirement certificates:** Generate upon credit sale with full provenance (capture unit ID, timestamp, verification proof, buyer information); certificate is PDF + machine-readable JSON-LD

3 API Gateway Layer

The API Gateway serves as the single entry point for all client requests, providing authentication, rate limiting, request validation, and observability.

Table 3: API Gateway Configuration

Component	Specification
Technology	Kong / AWS API Gateway
Authentication	JWT (partners, ops); API keys (corporate); OAuth2 (B2B); mTLS (internal, auditors)
Rate Limiting	100/min (partners); 1k/min (ops); 10k/min (corp API); 50k/min (internal)
SSL/TLS	TLS 1.3 only; auto renewal via ACM (90-day)
DDoS Protection	AWS Shield Advanced + WAF (OWASP Top 10, IP reputation, SQLi)
Request Validation	JSON schema (draft-07); limits: 1MB (partner), 10MB (corporate)
Logging	ELK stack; 90d hot storage, 1y cold (Glacier)
Caching	Redis cluster (3 shards, 2 replicas) for catalog, sessions, rate limits

3.1 API Versioning Strategy

- **URI versioning:** /v1/capture, /v2/capture. Major version changes indicate breaking changes (schema changes, removed endpoints). Minor version changes (bug fixes, performance) do not change URI.
- **Deprecation policy:** Announce 6 months in advance via email to all impacted API consumers; maintain backward compatibility for 12 months after deprecation announcement.
- **Current versions:** v1 (stable, 99.95% uptime over last 12 months), v2 (beta for new features including real-time WebSocket streaming, GraphQL support)

4 Microservices Layer

The microservices layer comprises loosely coupled services that can be scaled independently based on load. Services communicate via gRPC (synchronous, high-performance) for internal service-to-service calls and Kafka (asynchronous, durable) for event-driven workflows.

4.1 Microservices Summary

Table 4: Microservices Overview

Service	Stack	Priority	Function
Capture Monitoring	Python + MQTT + TF	H	IoT ingestion, efficiency calc, anomaly detection
Carbon Credit Engine	Java SB + Hyperledger	H	Credit issuance, blockchain audit, smart contracts
Credit Underwriting	Python + XGBoost	H	Loan scoring (50+ features), risk grading, GNPA
CaaS Billing	Node.js + Stripe + PG	H	Usage billing, payment collection, revenue rec
Predictive Maintenance	Python + Prophet + XGB	M	Solvent degradation (30d), failure prediction
Outgrow DAC Mgmt	Python Django + Celery	M	DAC monitoring, renewable tracking, offtake
Hedging Platform	Java + QuantLib + Kafka	M	Carbon derivatives, futures, margin, P&L
Transport Logistics	Go + OR-Tools	M	Pipeline allocation, ISO tank scheduling, tracking
Data Monetization	Python FastAPI + Spark	L	API sales, anonymized datasets, usage metering
Analytics & Reporting	Python + dbt + Superset	L	BI dashboards, ESG reports, executive KPIs
Notification Service	Node.js + Bull + Resend	L	Multi-channel alerts (email, SMS, push)
Blockchain Verifier	Go + Hyperledger	M	Credit audit trail, tamper-proof MRV, smart contracts

4.2 Service 1: Capture Monitoring Service

Handles real-time telemetry from CycloneCC capture units. This is the most critical service for operational reliability.

Technology: Python 3.11 + MQTT broker (Eclipse Mosquitto, clustered) + AWS IoT Core (device gateway) + TensorFlow Lite (edge inference).

Sensor count: 1,000+ currently → 100,000+ by FY30F.

Data pipeline: Sensor → MQTT (QoS 1) → AWS IoT Core (rule engine) → Kafka (topic: capture.telemetry.raw) → Python consumer → TimescaleDB.

Key functions:

- **Device provisioning:** OTA configuration (remotely update sampling rates, alert thresholds), firmware updates (signed binaries, rollback protection), device identity (X.509 certificates per device)
- **Data ingestion:** Real-time temperature (0.1°C resolution), pressure (0.01 bar), flow rate (0.1 m³/h), solvent concentration (0.1%), CO purity (0.5%), sampled every 10 seconds
- **Efficiency calculation:** Real-time capture efficiency (%) = (CO captured) / (CO in flue gas) with 5-second rolling average; compare against target (95%+)

- **Anomaly detection:** ML-based (Isolation Forest, autoencoder) detection of efficiency drops $\geq 5\%$ sustained for 30 minutes $\rightarrow$ trigger alert
- **Command dispatch:** Remote configuration of capture parameters (solvent injection rate, fan speed, valve positions); requires 2-factor authorization

Table 5: IoT Sensor Telemetry Schema

Field	Description
sensor_id	Unique device ID (X.509 CN), UUID v4
timestamp	UTC timestamp (ISO 8601), ms precision
temperature_c	Flue gas inlet temp ($^{\circ}\text{C}$), 20-400 $^{\circ}\text{C}$, $\pm 0.5^{\circ}\text{C}$
pressure_bar	Flue gas pressure (bar), 0.5-5 bar, ± 0.05
flow_rate_m3h	Flue gas flow rate (m^3/h STP), $\pm 2\%$
co2_concentration_pct	CO concentration (%), 0-30%, $\pm 0.5\%$
solvent_concentration_pct	Amine solvent (%), 10-40%, $\pm 0.5\%$
solvent_ph	Solvent pH, 7-14, ± 0.1
capture_efficiency_pct	Capture efficiency (%), 0-100%, $\pm 1\%$
energy_consumption_gj_tonne	Energy per tonne (GJ/tonne), from solvent regeneration
unit_id	CycloneCC unit ID (FK to units table)
plant_id	Partner facility ID (FK to partners table)
battery_voltage	Battery level (V), 0-5V range
rsi	Signal strength (dBm), -120 to -30

4.3 Service 2: Carbon Credit Engine

Handles automated credit issuance, verification, and monetization. This service is the core of Carbon Clean’s revenue model.

Technology: Java Spring Boot 3.x + Hyperledger Fabric 2.5 (blockchain) + PostgreSQL + Web3j (Ethereum integration for optional public chain anchoring).

Key functions:

- **Automated issuance:** Every tonne captured $\rightarrow$ verified against 3rd-party oracle (Chainlink, using IoT telemetry as proof) $\rightarrow$ credit issued via API to Verra/Gold Standard/CCTS. End-to-end latency: ≤ 48 hours (vs. 6+ months industry standard)
- **Blockchain audit trail:** Each credit has immutable record stored on Hyperledger Fabric (permissioned blockchain). Record includes: timestamp, capture unit ID, tonnes captured, verification proof hash, registry credit ID. This creates tamper-proof provenance.
- **Smart contract distribution:** Credit sale proceeds auto-split via smart contract (70% partner, 30% Carbon Clean) executed on settlement. Eliminates counterparty risk and manual reconciliation.
- **Real-time pricing:** Integration with voluntary carbon market exchanges (AirCarbon Exchange, Xpansiv CBL) via REST API. Price feeds update every minute; orders can be routed programmatically.

- **Retirement management:** Track credit retirements by buyer (corporate net-zero claims), generate retirement certificates (PDF + JSON-LD) with full provenance. Prevent double-counting via registry API.

4.4 Service 3: Credit Underwriting Engine (Carbon Clean Credit Engine)

Handles project loan underwriting for industrial partners. This service enables Carbon Clean’s financing facilitation business.

Technology: Python 3.11 + XGBoost (gradient boosting) + Scikit-learn + SHAP (model explainability) + PostgreSQL.

Training data: 5,000+ historical project loans (3 years of data) from Carbon Clean’s facilitated portfolio + external data (credit bureaus, industry benchmarks).

Model features (50+): Capture efficiency history (12 months), uptime percentage, solvent degradation rate, industry sector (cement/steel/refining), location (country, region), carbon price exposure (hedged vs. unhedged), financial statements (debt/equity, EBITDA margin), credit bureau score, project size (\$M), tenor (years).

Model performance: AUC-ROC 0.88, default prediction accuracy 82%, false positive rate 12%, false negative rate 6% (trained on 2023-2025 data, validated on 2026 data).

Key functions:

- **Partner onboarding:** Digital application with e-sign, financial statement upload (PDF parsing via OCR), credit bureau check (integration with Experian, Equifax, CIBIL)
- **Risk scoring:** ML model outputs probability of default (0-100%), risk grade (A1-D5), recommended interest rate spread (2-8% above cost of funds)
- **Loan structuring:** Tenor (5-7 years), interest rate (10-14%), repayment schedule (monthly credit revenue deduction), covenants (minimum capture efficiency 90%, maximum solvent degradation 2%/year)
- **Facility disbursement:** Via partner banks (Carbon Clean facilitates, does not lend own balance sheet). Automated drawdown request → bank API → funds to partner account within 48 hours.
- **Portfolio monitoring:** Real-time GNPA tracking (30, 60, 90+ days past due), early warning alerts (model flags partners with deteriorating efficiency or credit scores)

4.5 Service 4: CaaS Billing Service

Handles Capture-as-a-Service invoicing and collections.

Technology: Node.js 20.x + PostgreSQL + Stripe Connect (payment processing for multi-party splits) + ACH/SEPA/UPI gateways.

Key functions:

- **Usage-based billing:** Monthly invoice based on tonnes captured (CaaS fee \$5-10/tonne, tiered by volume). Invoice generated on 1st of month, due net 30.
- **Credit share reconciliation:** Calculate 30% share of credit revenue (from Carbon Credit Engine), net against CaaS fees. Partners receive net invoice (CaaS fee - credit share) or refund if credit share exceeds CaaS fee.
- **Automated collections:** ACH (US), SEPA (EU), UPI (India) auto-debit for CaaS fees. Partners enroll via Stripe Connect (bank account verification). Success rate: 94% (first attempt).
- **Late payment handling:** Grace period: 15 days. Late fee: 1.5%/month (18% APR). Service suspension: after 60 days past due (with 7-day notice). Reinstatement: full payment + reconnection fee (\$5,000).
- **Tax compliance:** VAT/GST calculation by jurisdiction (EU: reverse charge, India: 18% GST, US: sales tax by state), e-invoicing (PEPPOL for EU, IRN for India), tax report generation (quarterly filings).

4.6 Service 5: Predictive Maintenance Service

Predicts solvent degradation and component failures to minimize downtime.

Technology: Python + Prophet (Facebook time-series) + XGBoost + PostgreSQL + Grafana (visualization).

Key functions:

- **Solvent degradation prediction:** Forecast replacement date with 90% accuracy (30-day horizon). Model features: solvent concentration trend, pH trend, temperature history, capture efficiency drop. Output: Replace solvent in 14 days (confidence 92%)
- ..
- **Component failure prediction :** *Pumps(bearingwear, seal leakage), valves(stiction, seat wear), vibration analysis(frequency domain), pressure delta, flow rate deviation.* Output : probability of failure
- **Maintenance scheduling:** Optimize dispatch of field service engineers to minimize travel cost (vehicle routing problem). Prioritize high-risk units (failure probability > 50%). Generate work orders with required parts list.
- **Spare parts inventory optimization:** Predictive stocking based on failure forecasts. Safety stock calculation (service level 95%, lead time 7 days). Auto-reorder when inventory falls below reorder point.

4.7 Service 6: Outgrow DAC Management Service

Manages Direct Air Capture (DAC) units (post-FY28 launch).

Technology: Python Django + Celery (async task queue) + PostgreSQL + Redis (rate limiting).

Key functions:

- **DAC unit monitoring:** Lower capture efficiency (400-500 ppm CO concentration vs. 5-15% for point-source). Typical efficiency: 75-85% (vs. 95%+ for point-source). Higher energy consumption: 8-12 GJ/tonne (vs. 2.5-3.5 GJ/tonne for point-source).
- **Energy source integration:** Track renewable energy fraction (solar, wind, waste heat) for green DAC credits. Integration with weather APIs (solar irradiance forecasting) and utility meters (real-time energy mix).
- **Carbon removal buyer offtake:** Manage 10-year agreements with corporate buyers (Microsoft \$1B commitment, Stripe \$500M, Airbus \$300M). Track delivery against annual quotas, generate offtake certificates.
- **Consumer subscription management:** Handle individual carbon removal subscribers (monthly recurring billing via Stripe). Track tonnes removed per subscriber, generate personalized impact reports.

4.8 Service 7: Hedging Platform

Manages carbon price hedging products (Strategy 4 from Revenue Expansion Playbook).

Technology: Java 21 + QuantLib (financial library for pricing derivatives) + Kafka (market data feeds) + PostgreSQL.

Key functions:

- **Price guarantee contracts:** Manage portfolio of guaranteed minimum price contracts (€50/tonne floor for 5 years). Contract value: \$10-100M per partner. Track mark-to-market daily, calculate exposure.
- **Hedge execution:** Interface with EU ETS futures exchanges (ICE Futures Europe, EEX) via FIX protocol (Financial Information eXchange). Execute hedge trades (futures, options, swaps) to offset guarantee exposure.
- **Margin management:** Calculate variation margin daily based on futures price movements. Post margin to exchange via clearing broker (Goldman Sachs, JP Morgan). Margin call alerts when cash balance $\leq$ threshold.
- **Reinsurance:** Automatic hedging of tail risk (e.g., carbon price collapse to €10/tonne) with Swiss Re/Munich Re. Premium: 2-3% of notional value. Triggers when price $\leq$ €20/tonne for 30 consecutive days.

- **P&L attribution:** Track hedge effectiveness (ratio of hedge P&L to exposure P&L), P&L by partner, by contract, by hedging instrument. Generate quarterly hedge effectiveness reports for accounting (IFRS 9, hedge accounting).

4.9 Service 8: Transport Logistics Service

Manages CO transport via pipeline and ISO tank containers.

Technology: Go 1.21 + Google OR-Tools (vehicle routing problem solver) + PostgreSQL (spatial via PostGIS) + Redis (real-time tracking).

Key functions:

- **Pipeline capacity management:** Allocate pipeline slots (Enterprise Products in US Gulf Coast, Pembina in Canada). Real-time capacity monitoring via SCADA integration. Automated scheduling: minimize idle time, maximize throughput.
- **ISO tank scheduling:** Optimize shipping routes for liquid CO (truck, rail, vessel). Vehicle routing problem with time windows, capacity constraints (20-40 tonnes per ISO tank), and delivery priority (sequestration deadlines).
- **Delivery tracking:** Real-time GPS for ISO tank shipments (update every 5 minutes). Geofencing for delivery confirmation (trigger when within 100m of destination). ETA updates via SMS/email to recipient.
- **Storage integration:** Interface with sequestration sites (Carbfix in Iceland, 1Point-Five in Texas). API for injection scheduling, real-time injection rate monitoring, cumulative tonnes stored verification.

4.10 Service 9: Data Monetization Service

Sells anonymized, aggregated data to enterprise customers via API.

Technology: Python FastAPI + Apache Spark (data aggregation at scale) + Snowflake (data source).

Data volume: 10 TB raw data per month (FY26E) → 100 TB by FY30F.

Anonymization: Remove PII (partner name, contact, location precision $\geq$ district level), aggregate at industry/region/month level (minimum 100 partners per cohort), add statistical noise (differential privacy, epsilon=0.5).

Datasets available (priced by subscription tier):

- **Tier 1 (\$10,000/month):** Capture efficiency benchmarks by industry (cement, steel, refining), by region (NA, EU, Asia), by unit size (100-500 tpd, 500-1000 tpd)
- **Tier 2 (\$25,000/month):** Solvent degradation curves (real-world degradation data, anonymized), energy consumption benchmarks (GJ/tonne by technology type, by region)

- **Tier 3 (\$50,000/month):** Carbon credit price trends (historical tick data, 5-minute resolution), predictive forecasts (30-day horizon, 85% accuracy), supply-demand models

4.11 Service 10: Analytics & Reporting Service

Provides business intelligence and ESG reporting dashboards.

Technology: Apache Superset (open-source BI) / Power BI (enterprise) + dbt (data transformation) + Snowflake (data warehouse).

Key dashboards:

- **Executive dashboard** (real-time, cached 5 minutes): Revenue (daily, MTD, YTD), gross margin (%), ROE (%), ROA (%), AUM growth (MoM), GNPA trends (30/60/90+ days)
- **Operations dashboard** (real-time): Capture efficiency by unit (top/bottom 10), uptime (%) by region, energy intensity (GJ/tonne) by industry, solvent degradation rate (%/month)
- **Credit dashboard** (daily refresh): Loan application volume (funnel: submitted → underwriting → approved → disbursed), approval rate (% by risk grade), GNPA by region, by industry, by vintage
- **ESG dashboard** (monthly refresh): Tonnes captured (cumulative, by year), avoided emissions vs. baseline, Sustainable Development Goals (SDGs) contribution (SDG 13: Climate Action), ESG rating (MSCI, Sustainalytics)
- **Partner portal dashboards** (real-time, per-partner): Real-time capture performance (tonnes captured today, efficiency %), credit wallet (credits issued, sold, retired, balance), maintenance alerts (predicted replacement dates)

4.12 Service 11: Notification Service (Cross-cutting)

Handles all outbound communications across channels with guaranteed delivery.

Technology: Node.js + Bull (Redis-backed job queue) + channel adapters (Resend for email, Twilio for SMS, Expo for push, Webhook for API).

Channels:

- **Email** (primary for corporate): Invoices, reports, credit issuance notifications, audit trail exports
- **SMS** (field service): Maintenance alerts (critical: efficiency drop >10%), one-time passwords (2FA)
- **In-app push** (partner portal): Real-time credit sale notifications, payment confirmations

- **Webhook** (API): Programmatic notifications for corporate buyers (credit issuance, retirement events)

Queue management: Priority queues (high: service alerts, credit issuance; medium: invoice reminders, payment confirmations; low: marketing newsletters, product updates). Dead Letter Queue (DLQ) for failed deliveries (retry with exponential backoff: 1min, 5min, 30min, 2hr, 6hr, 24hr → then manual intervention).

5 Data Layer

The data layer consists of multiple storage systems optimized for different use cases (polyglot persistence), each chosen for specific access patterns and consistency requirements.

5.1 Operational Databases

Table 6: Operational Databases

Database	Technology	Purpose	Size (FY30F)
Primary OLTP	PostgreSQL 15	Partners, loans, credits, invoices, user sessions (ACID compliant)	20 TB
Time-series	TimescaleDB (PostgreSQL ext.)	IoT telemetry: temperature, pressure, efficiency (100M+ readings/day)	50 TB
Blockchain	Hyperledger Fabric 2.5	Credit audit trail, immutable records, append-only, cryptographically linked	10 TB
Caching	Redis Enterprise (cluster)	Sessions, rate limits, real-time credit prices, catalog cache (1ms latency)	500 GB
Search	Elasticsearch 8.x	Partner search, audit logs, full-text search on maintenance records	5 TB

5.2 Data Warehouse

Technology: Snowflake (primary) with auto-scaling (X-Small to 3X-Large warehouses). Amazon Redshift (legacy, being migrated).

Data sources: All microservices stream data to Kafka (50+ topics) → transformed via dbt (100+ models, SQL-based) → loaded to Snowflake.

Data volume: 10 TB raw data per month (FY26E) → 100 TB by FY30F (10x growth).

Refresh frequency: Real-time (streaming via Snowpipe) for operational dashboards; daily for analytical reports (3 AM UTC, 2 hour batch window).

Schema: Star schema with fact tables (`capture_fact`, `credit_fact`, `loan_fact`, `invoice_fact`) and dimension tables (`partner_dim`, `date_dim`, `unit_dim`, `industry_dim`, `region_dim`). Slowly changing dimensions (SCD Type 2) for partner attribute history.

5.3 Data Lake

Technology: Amazon S3 (with intelligent-tiering) + Glacier Deep Archive for cold storage (retrieval time 12-48 hours).

Content:

- **Raw IoT telemetry:** Retained 30 days in S3 Standard (for real-time replay), 7 years in Glacier Deep Archive (regulatory requirement for MRV audit)
- **Partner KYC documents:** Encrypted at rest (AES-256), retained 7 years post-last-transaction (RBI NBFC requirement)
- **Carbon credit audit trail:** Retained indefinitely (Verra/Gold Standard requirement)
- **System logs** (application, access, audit): Retained 30 days in S3 Standard (hot), 90 days in S3 Glacier Instant Retrieval (warm), 1 year in Glacier Deep Archive (cold)
- **ML model artifacts:** Versioned (model registry), retained indefinitely (for reproducibility, regulatory audit)

5.4 Data Pipeline Architecture

- **Ingestion:** Kafka (50+ topics, 9 brokers, `kafka.m5.2xlarge`) for real-time streaming from all microservices. Topics partitioned by service (e.g., `capture.telemetry` (32 partitions), `credit.issuance` (8 partitions))
- **Processing:** Apache Spark (EMR) for batch (nightly aggregations) + Kafka Streams for stream (real-time alerts, efficiency calculations)
- **Transformation:** dbt (100+ models, SQL-based) for data warehouse transformations. Models tested with dbt test (unique, not-null, relationships, accepted values)
- **Orchestration:** Apache Airflow (MWAA) for scheduled jobs (DAGs for ETL, report generation, ML retraining, data quality checks)
- **Quality:** Great Expectations for data validation (automated testing of data quality rules: freshness, completeness, uniqueness, referential integrity)

5.5 Data Governance & Security

- **Encryption:** AES-256 at rest (S3, RDS, Snowflake, EBS volumes); TLS 1.3 in transit (all services, API Gateway, internal service mesh)
- **Access control:** Row-level security (RLS) in PostgreSQL (partners see only their own data); column-level security in Snowflake (PII columns redacted for non-authorized roles); IAM roles for AWS services (least privilege)
- **Data classification:** Public (efficiency benchmarks, aggregated market data), Internal (operational metrics, system logs), Confidential (partner PII: name, contact, financials), Restricted (loan details, credit pricing, trade secrets)
- **Compliance:** Verra/Gold Standard MRV (Measurement, Reporting, Verification), EU ETS (Monitoring and Reporting Regulation), US 45Q (IRS attestation), ISO 27001 (planned FY28), SOC 2 Type II (planned FY27)
- **Backup:** Automated daily snapshots (RDS, EBS, Snowflake) retained 30 days; cross-region replication to ap-southeast-1 (Singapore) for DR; point-in-time recovery (PITR) for PostgreSQL (7-day retention)

6 External Integrations

6.1 Carbon Registries

Table 7: Carbon Registry Integrations

Registry	Purpose	Volume (FY30F)
Verra (Verified Carbon Standard)	Credit issuance, MRV compliance, retirement certification. Most widely used registry for industrial capture.	40M tonnes/year
Gold Standard	Premium credit issuance (higher price, SDG co-benefits). Requires additional verification for sustainable development impacts.	10M tonnes/year
India CCTS (Carbon Credit Trading Scheme)	Domestic credit issuance for India operations. Government-run, mandatory for certain sectors.	5M tonnes/year

6.2 Financial Exchanges & Payment Gateways

Table 8: Financial Integrations

Partner	Purpose	Volume (FY30F)
ICE (Intercontinental Exchange)	EU ETS futures hedging (primary exchange for carbon futures). FIX protocol integration.	\$500M/year
EEX (European Energy Exchange)	EU ETS options (puts and calls for hedging). REST API + FIX.	\$200M/year
Stripe Connect	CaaS billing, credit sales, multi-party splits (70/30 partner/Carbon Clean). Supports ACH, SEPA, UPI, cards.	\$1.28B/year
Coinbase Prime	Crypto credit sales (premium pricing for Bitcoin purchases). Institutional custody and trading.	\$50M/year
Swiss Re / Munich Re	Tail risk reinsurance (hedges carbon price collapse). Parametric triggers based on EU ETS price.	\$100M coverage

6.3 External APIs

- **Weather APIs** (OpenWeather, Tomorrow.io): Solar irradiance forecasting for DAC units (48-hour horizon, 1-hour resolution). Free tier for non-commercial, enterprise tier for commercial use (\$2,000/month).
- **Map APIs** (Google Maps Platform, Mapbox): Logistics routing (distance matrix, traffic-aware ETA), geocoding (address → coordinates), reverse geocoding (coordinates → address for delivery confirmation). Cost: \$0.005/request (bulk discount).
- **AWS IoT Core**: IoT sensor connectivity (MQTT), device shadow (state synchronization), fleet provisioning (bulk device onboarding), OTA updates. Cost: \$1.25 per million messages.
- **Datadog**: Infrastructure monitoring (CPU, memory, network, custom metrics) and alerting (PagerDuty integration). Cost: \$15 per host per month.
- **Sentry**: Error tracking (unhandled exceptions, performance monitoring), crash reporting (mobile apps). Cost: \$26 per month (Team plan).

7 Scalability & Performance

7.1 Scaling Strategy

The architecture is designed to scale horizontally to handle 5x growth by FY30F. Each microservice can be scaled independently based on its specific load patterns.

Table 9: Horizontal Scaling Targets by FY30F

Component	FY26E	FY30F	Growth
Industrial partners	450	1,200	2.7x
IoT sensors deployed	1,000	100,000	100x
Daily IoT data points	1,000,000	100,000,000	100x
Daily API requests	500,000	10,000,000	20x
Data warehouse size (TB)	10	100	10x
Daily credit issuance events	10,000	200,000	20x
Concurrent WebSocket connections	1,000	10,000	10x

7.2 Auto-scaling Configuration

Orchestration: Amazon EKS (Elastic Kubernetes Service) with Horizontal Pod Autoscaler (HPA) for pod-level scaling and Cluster Autoscaler for node-level scaling.

Trigger conditions:

- CPU utilization >70% for 2 minutes
- Memory utilization >80% for 2 minutes
- Custom metric: requests per second >1,000 per pod (API gateway)
- Custom metric: Kafka consumer lag >10,000 messages (capture monitoring)

Scaling behavior: Add 50% more pods per scale-up event; cooldown period 3 minutes (scale up), 10 minutes (scale down).

Node groups (EC2 instance types):

- **Compute-optimized** (c5.large, c5.xlarge): Credit engine, billing, analytics. High CPU:memory ratio (1:2). Spot instances for batch processing.
- **Memory-optimized** (r5.large, r5.xlarge): PostgreSQL, Redis, Elasticsearch. High memory:CPU ratio (1:8). On-demand only (stateful).
- **GPU-optimized** (g4dn.xlarge): ML inference (efficiency prediction, degradation forecasting). NVIDIA T4 GPU (16GB). Spot for training, on-demand for inference.
- **IoT-optimized** (m5.large): MQTT broker, telemetry ingestion. Balanced CPU/memory. On-demand for critical ingestion, spot for processing.

7.3 Performance SLAs

Table 10: Performance Service Level Agreements (SLAs)

Service	Target (p95)	Current (p95)
Capture telemetry ingestion (sensor to database)	≤ 500 ms	280 ms
Credit decision (automated underwriting)	≤ 5 sec	3.0 sec
Credit issuance (Verra API end-to-end)	≤ 2 sec	1.2 sec
Partner portal dashboard load (initial)	≤ 2 sec	1.5 sec
API data access (aggregated, cached)	≤ 1 sec	0.6 sec
Maintenance alert generation (trigger to notification)	≤ 10 sec	4 sec
Hedge execution (futures order placement)	≤ 1 sec	0.5 sec
Route optimization (1000 nodes, VRP)	≤ 30 sec	15 sec
Credit audit trail query (blockchain, 1 year range)	≤ 5 sec	3.2 sec

8 Deployment Architecture

8.1 AWS Infrastructure

Table 11: AWS Infrastructure Components

Component	Configuration
Compute (K8s)	EKS — 20-100 nodes (c5.large, c5.xlarge, r5.large, g4dn.xlarge, m5.large)
Database	RDS PostgreSQL 15 Multi-AZ, db.r5.2xlarge (16 vCPU, 64 GB RAM, 10,000 IOPS)
Time-series	TimescaleDB managed on RDS (same instance as PostgreSQL), 50 TB storage (provisioned IOPS)
Caching	ElastiCache for Redis cluster (3 shards, 2 replicas each, cache.r5.xlarge)
Object storage	S3 — 100 TB standard (intelligent-tiering), 500 TB Glacier Deep Archive
Data warehouse	Snowflake — X-Small to 3X-Large warehouses (auto-scaling), multi-cluster
Streaming	MSK (Managed Streaming for Kafka) — 9 brokers (kafka.m5.2xlarge), 30 TB storage
CDN	CloudFront — 25+ edge locations globally (15 in India, 5 in US, 5 in Europe)
Blockchain	Amazon Managed Blockchain (Hyperledger Fabric) — 4 peers, 3 orderers
DNS	Route 53 (latency-based routing, failover to DR region)

8.2 Multi-Region Deployment

Table 12: AWS Regions

Region	Purpose
US East (N. Virginia) - us-east-1	Primary region (North America operations). Hosts US partners, 45Q compliance data.
EU West (London) - eu-west-2	Primary region (Europe operations). Hosts EU partners, EU ETS compliance data. GDPR compliant.
Asia Pacific (Mumbai) - ap-south-1	Primary region (India & SE Asia operations). Hosts India partners, CCTS compliance data. Data localization for India.
Asia Pacific (Singapore) - ap-southeast-1	Disaster Recovery region (cross-region replication for all three primary regions). Warm standby.

8.3 Disaster Recovery

Table 13: Disaster Recovery Objectives

Metric	Target
RTO (Recovery Time Objective)	2 hours (critical: 1 hour for credit issuance, capture monitoring)
RPO (Recovery Point Objective)	15 minutes (critical data: 5 minutes for credit book, partner profiles)
Backup frequency	Daily automated snapshots (RDS, EBS); continuous S3 replication (CRR)
DR testing	Quarterly (simulated failover, tabletop exercise); annual (full failover test)

DR Region: Singapore (ap-southeast-1).

Replication strategy:

- Cross-region replication for critical data (credit book, partner profiles, credit issuance records, IoT telemetry aggregates)
- RDS read replica (async replication, lag ≤ 5 seconds typical, ≤ 30 seconds during peak)
- S3 cross-region replication (CRR) for KYC documents, audit logs, ML models

- Warm standby: EKS cluster with minimum nodes (20% of production capacity) running in DR region, ready to scale up on failover
- Route 53 health checks (30-second interval, 3 consecutive failures triggers failover)
- Manual failover (super admin approval required) for data integrity validation; automated failover for stateless services only

8.4 CI/CD Pipeline

- **Source control:** GitHub Enterprise (monorepo with 15+ microservices). Branch protection: main branch requires PR approval (2 reviewers), passing CI checks, CLA signed.
- **CI:** GitHub Actions (build, test, lint, security scan). Build matrix: Node.js 20, Python 3.11, Java 21, Go 1.21. Security scan: Snyk (dependency vulnerabilities), SonarQube (code quality), Trivy (container images).
- **CD:** ArgoCD (GitOps, automated deployment to EKS). Sync every 5 minutes. Manual sync for production (require approval). Rollback: automated (health check failure) or manual (one-click via ArgoCD UI).
- **Environments:** dev (feature branches) → staging (main branch, post-merge) → production (tagged releases, e.g., v2.3.0). Canary deployment for production: 5% traffic (monitor 1 hour) → 20% (monitor 2 hours) → 100%.
- **Build frequency:** 50+ builds per day (dev/staging); 2-3 production releases per week (Wednesday, Friday, with hotfixes as needed).
- **Testing:** Unit tests (80% coverage required), integration tests (service-to-service via testcontainers), end-to-end tests (Cypress for UI, Postman for API), performance tests (k6, 10K concurrent users).

9 Offline-First Design

Given that 20-30% of industrial partner sites may have intermittent connectivity (remote cement plants in Rajasthan, offshore platforms in North Sea, mines in interior China), Carbon Clean implements offline-first design principles to ensure operational continuity.

9.1 Offline Capabilities by Component

Table 14: Offline Functionality Matrix

Component		Offline	Max tion	Dura-	Sync Strategy
Capture (sensors)	telemetry	Yes	30 days		Batch upload (MQTT persistent session, QoS 1)
Field service app		Yes	7 days		Delta sync on connectivity (WatermelonDB, optimistic updates)
Maintenance reporting		Yes	7 days		Upload with photo evidence (exponential backoff retry)
Partner portal (web)		Partial	4 hours		Stale-while-revalidate (Service Worker cache, background sync)
Credit (draft)	application	Yes	30 days		Submit online (idempotent: retry with same idempotency key)
Dashboard analytics		No (read-only)	N/A		Requires connectivity; cached data for 1 hour

9.2 Local Storage Architecture

- **Field service app:** SQLite via WatermelonDB (up to 1 GB) for partner data (cached), service tickets (pending/completed), offline forms (checklists, photos). Sync: delta updates (only changed records) when connectivity restored.
- **IoT sensors** (CycloneCC edge gateway): Flash storage (64 MB, expandable via SD card) for telemetry; store-and-forward for up to 30 days (100,000 readings, 5-minute resolution). Sync: MQTT persistent session (clean session=false), QoS 1 (at-least-once). Replay: broker stores messages until device reconnects.
- **Edge gateway** (for DAC units in remote locations): Local SQLite + file storage (1 GB); computes capture efficiency offline (using cached ML model), stores results; syncs every 6 hours via satellite internet (Starlink) or cellular (4G failover).

9.3 Sync Conflict Resolution

Table 15: Sync Conflict Resolution

Scenario	Resolution
Field service report (offline edit vs. cloud edit)	Last-write-wins (timestamp) with manual merge option for critical fields
Maintenance checklist (offline vs. cloud)	Server wins for status fields; client wins for notes (append)
Capture telemetry (duplicate readings)	Idempotency key (sensor_id + timestamp) prevents duplicates
Partner profile update (offline vs. cloud)	Server wins (latest version from CRM)
Credit application draft (offline submit)	Idempotency key (farmer_id + timestamp) prevents duplicate applications
Delivery proof photo (offline upload)	Server wins; client retries with backoff (1min, 5min, 30min, 2hr)

10 Security Architecture

10.1 Authentication & Authorization

Table 16: Authentication Methods by User Type

User Type	Method	Controls
Industrial Partners	JWT + password	2FA optional; 8 hr timeout; 5 failed attempts → 15 min lockout
Carbon Credit Buyers	API keys + IP whitelist	Key rotation 90 days; scope-limited permissions (read/write/admin)
Field Service Engineers	JWT + MFA (TOTP)	12 hr timeout; device attestation (only registered devices)
Corporate API Users	API keys + OAuth2 (client credentials)	Scope-limited tokens; rate limits by key; IP restrictions (optional)
Admins	MFA (TOTP) mandatory	2 hr timeout; audit logs (all actions); break-glass access (logged, requires justification)
Internal Services	mTLS (service mesh, Istio)	SPIFFE identities; auto cert rotation (24 hours); mutual authentication
Registry Auditors (Verra, Gold Standard)	OAuth2 (authorization code) + client certs	Read-only access to audit trail; session timeout 2 hours

10.2 RBAC Roles and Permissions

Table 17: RBAC Roles (8 roles)

Role	Permissions
Industrial Partner	View own capture data, CaaS billing, credit wallet, maintenance scheduling; request service; download compliance reports
Carbon Credit Buyer	Purchase credits (POST /credits/purchase), view portfolio (GET /portfolio), download retirement certificates; no access to others' data
Field Service Engineer	View assigned tickets, update service status (PENDING → IN_PROGRESS → COMPLETED), order parts (with approval for $\geq$ \$500)
Credit Underwriter	Assess applications (POST /underwriting/evaluate), view credit risk (GET /risk/{partner_id}), approve/decline loans (POST /loans/{id}/status)
Partner Manager	Manage partner relationships (contracts, pricing, credit limits); view partner portfolio (GET /partners/{id}/portfolio)
Admin	User management (create/disable roles), system configuration (feature flags, rate limits), report access (all reports), audit log viewer
Auditor	Read-only access to audit trail (GET /audit/*), MRV data (GET /mrv/*), credit issuance logs; no write access
Super Admin	Full system access; audit log review; role assignment; break-glass emergency access; DR failover authorization

10.3 Security Controls Summary

Table 18: Security Controls

Control	Implementation
Authentication	JWT (RS256 signed); MFA (TOTP, Google Authenticator/Authy); API keys (UUIDv4, hashed at rest); mTLS (service mesh)
Authorization	RBAC (8 roles); ABAC for fine-grained (region, partner tier, industry vertical)
API Security	JWT validation at gateway (Kong); rate limiting by role; input validation (JSON schema, regex)
Data Encryption (Rest)	AES-256 (RDS, S3, Snowflake, Elasticsearch, EBS); KMS key rotation 365 days
Data Encryption (Transit)	TLS 1.3 only (TLS 1.2 rejected); HSTS preload; mTLS for service-to-service (Istio)
Secrets Management	AWS Secrets Manager (API keys, DB credentials, certs, Stripe keys); auto rotation 30-90 days
Vulnerability Scanning	Weekly automated (AWS Inspector, Snyk, Trivy); critical CVEs patched within 48 hours
Penetration Testing	Annual third-party (CERT-In for India, CREST for US/Europe); external APIs + mobile apps
Audit Logging	All API calls logged (user ID, timestamp, IP, endpoint, request ID); immutable write-once storage
IAM	Least privilege principle; temporary credentials (STS) for EC2/EKS; quarterly access reviews
WAF	AWS WAF (OWASP Top 10, IP reputation, SQLi prevention, XSS protection, rate-based rules)

10.4 IoT Device Security (Capture Unit Sensors)

- **Device identity:** X.509 certificates (per device) issued by AWS IoT Core (private CA). Certificate validity: 5 years. Auto-renewal: 30 days before expiry.
- **Mutual TLS (mTLS):** Device authenticates to cloud (certificate + private key); cloud authenticates to device (server certificate). Prevents man-in-the-middle (MITM) attacks.
- **Firmware signing:** OTA updates only with signed binaries (RSA-2048, private key)

offline, public key embedded). Rollback protection (reject older versions). Signed manifest includes hash of all files.

- **Tamper detection:** Physical tamper switch (triggered when enclosure opened). Event logged to secure element, alert sent to cloud. Erase keys after 3 tamper events (anti-forensics).
- **Data validation:** Anomaly detection for impossible values (e.g., capture efficiency $\geq 100\%$, temperature $\geq 500^\circ\text{C}$ in chilled environment, pressure negative). Out-of-range values rejected, alert triggered.
- **Secure boot:** U-Boot verified boot (signatures on bootloader, kernel, initramfs). Hardware root of trust (Trusted Platform Module 2.0).

10.5 Compliance Mapping

Table 19: Compliance Mapping (Verra, Gold Standard, EU ETS, 45Q, RBI, DPDP)

Requirement	Verra	EU ETS	45Q	RBI	DPDP
Data localization (in-region)	—		—		
MRV (Measurement, Reporting, Verification)				—	—
Third-party audits (annual)					—
Audit trail (all transactions)					
Blockchain verification (optional)		—	—	—	—
Independent validation				—	—
Retirement certificates		—		—	—
Consent management	—	—	—	—	
Right to be forgotten	—	—	—	—	
Data breach notification (72 hours)	—	—	—		

11 Monitoring & Observability

11.1 Monitoring Stack

Table 20: Monitoring Tools

Tool	Purpose
Datadog	Infrastructure metrics (CPU, memory, network, disk I/O) with 1-minute granularity
Prometheus + Grafana	Application metrics (latency, errors, request rate, business KPIs) with 15-second granularity
ELK Stack (Elasticsearch, Logstash, Kibana)	Centralized logging (app logs, audit logs, access logs) with 30-day retention
AWS X-Ray	Distributed tracing for microservices (end-to-end request tracing, latency breakdown)
Sentry	Error tracking (unhandled exceptions), crash reporting (mobile apps), performance monitoring (slow transactions)
PagerDuty	On-call alerting, incident management (escalation policies, post-mortems), SLO tracking
AWS CloudTrail	AWS API audit logging (who created/modified/deleted resources) with 90-day retention
AWS Config	Resource compliance monitoring (ensure resources follow security policies, e.g., S3 not public)

11.2 Key Metrics to Monitor

11.2.1 Business KPIs (Dashboard - 5 min refresh)

- Revenue (daily, weekly, monthly by region: NA, EU, India, SE Asia), gross margin (%), GPV (Gross Processing Value)
- Tonnes captured (daily, MTD, YTD) by industry (cement, steel, refining), by region, by partner tier
- Credit issuance volume (daily, MTD), credit price trends (spot vs. futures), average selling price by buyer tier
- Partner onboarding rate (daily) and retention (94% target, 90% acceptable, ¡85% alert)

- IoT sensor health (% online, battery $\geq$ 20% threshold, signal strength $\geq$ -90dBm target)
- Capture efficiency (%) by unit type (CycloneCC v2 vs. v3), by region, by solvent batch
- GNPA trends (30/60/90+ days past due), loan disbursement volume (daily, MTD, YTD)
- Maintenance cost per tonne captured (\$/tonne), mean time between failures (MTBF, hours)

11.2.2 Technical KPIs (Dashboard - real-time)

- API p95 latency by endpoint (target: <1 sec for /capture, <5 sec for /underwriting, <2 sec for /credit)
- Error rate (HTTP 4xx client errors, 5xx server errors) — target <0.1% (99.9% availability), <0.5% alert
- Credit engine inference latency (p95) — target <400 ms, <600 ms alert
- IoT data ingestion lag (device timestamp $\rightarrow$ database commit) — target <1 second, <5 seconds alert
- Database connection pool utilization (% of max connections) — target <80%, >90% alert
- Kafka consumer lag (by consumer group, by topic partition) — target <10,000 messages, >50,000 alert
- Pod restart count (by service, by reason: OOMKilled, LivenessProbe failure) — target 0 per 24h, >5 alert
- Node CPU/memory utilization (by node group) — target <70%, >85% alert
- Blockchain consensus latency (time to commit a transaction) — target <2 seconds, <5 seconds alert

11.3 Alerting Rules

- **Critical (P0 - page on-call engineer within 2 minutes):** API error rate (5xx) >1% for 5 minutes; Credit issuance failure >10 minutes; Capture telemetry ingestion lag >15 minutes; Database unreachable >1 minute
- **High (P1 - SMS to operations manager within 5 minutes):** Capture efficiency drop >10% for 30 minutes (sustained); Solvent degradation accelerating (>3%/month); GNPA >3% (target 1.8%); Partner churn rate >10% in 30 days

- **Medium (P2 - Slack to engineering team within 10 minutes):** Database connection pool >80% for 10 minutes; Kafka consumer lag >50,000 for 5 minutes; Pod restart count >5 in 1 hour; SSL certificate expiry <30 days
- **Low (P3 - email to field ops within 1 hour):** IoT sensor offline >24 hours; Battery level <20%; API rate limit approaching (>80% of limit); Backup failure (RDS snapshot, S3 replication)
- **Credit-specific (P0 - page credit ops):** Credit issuance delay >4 hours (registry API down); Retirement certificate generation failure >30 minutes; Smart contract execution failure (blockchain)

12 Cost Optimization

12.1 Current & Projected Cloud Spend

Table 21: Cloud Spend Projection (\$M)

Category	FY26E	FY28F	FY30F
Compute (EKS, EC2, Lambda, Fargate)	1.5	3.0	6.0
Storage (RDS, S3, Snowflake, EBS, Glacier)	0.8	1.5	3.0
IoT connectivity (AWS IoT Core, MQTT messages)	0.3	1.0	2.5
Blockchain (Managed Blockchain Hyperledger)	0.1	0.3	0.8
Data transfer (in/out, CloudFront, cross-region)	0.2	0.5	1.0
Third-party APIs (registries, exchanges, maps, weather)	0.5	1.0	2.0
Total Cloud Spend	3.4	7.3	15.3
<i>As % of Revenue</i>	<i>1.2%</i>	<i>1.1%</i>	<i>1.2%</i>

Note: Cloud spend as % of revenue remains stable at 1.2%, reflecting efficient scaling despite 4.6x revenue growth.

12.2 Optimization Strategies

1. **Reserved Instances (RI):** Purchase 3-year RIs (All Upfront) for stable workloads (control plane, databases, Redis, Kafka brokers) → 35% savings vs. On-Demand (approximately \$1.2M/year by FY30F). Use AWS Compute Optimizer for recommendations.
2. **Spot instances:** Use for non-production (staging, dev) and batch processing (data aggregation, ML training, reporting) → 65% savings vs. On-Demand. Use node groups with mixed Spot/On-Demand, Spot termination handling (drain nodes 2 min before termination).

3. **S3 lifecycle policies:** Transition IoT telemetry to Glacier Instant Retrieval after 30 days (S3 Standard → S3 Standard-IA → Glacier Instant Retrieval) → 80% storage cost reduction. Data older than 1 year to Glacier Deep Archive (92% reduction). Use S3 Intelligent-Tiering for unknown access patterns.
4. **Data compression:** Compress IoT telemetry (gzip level 6, 70% compression ratio) before upload → 50% reduction in data transfer costs and S3 storage. Use Parquet format (columnar, 80% compression) for data lake.
5. **Serverless for low-traffic:** Use Lambda for infrequent operations (report generation (once/hour), data exports (once/day), scheduled jobs (cron)). Pay-per-execution (vs. idle EC2). Use for: daily report generation, weekly partner summary emails, monthly invoice generation.
6. **Committed use discounts (CUD):** For Snowflake (pre-purchase credits for 1-3 years), Datadog (annual commit), Stripe (volume discounts > \$1M/year) → 25-30% savings vs. pay-as-you-go.
7. **Graviton/ARM-based instances:** Migrate compute workloads (EKS node groups) to AWS Graviton3 (c7g/r7g instances) for Python/Go/Node.js services → 20% price/performance improvement. Java workloads require testing (GraalVM native image).
8. **Network optimization:** Use CloudFront for static assets (web portal, mobile app downloads) → 70% reduction in origin fetches. Use S3 Transfer Acceleration for large uploads (IoT telemetry, KYC documents). VPC endpoints for S3, DynamoDB (no NAT gateway charges).

13 Conclusion

The Carbon Clean system architecture is designed to support the company’s ambitious growth targets from \$280M (FY26E) to \$1.28B+ (FY30F) while maintaining high availability (99.95% uptime target), sub-second response times for critical operations (capture monitoring, credit issuance), and full compliance with Verra/Gold Standard MRV, EU ETS, US 45Q, and emerging carbon market regulations.

13.1 Key Architectural Decisions Summary

Table 22: Key Architectural Decisions & Rationale

Decision		Rationale
Microservices over monolithic		Independent scaling (capture monitoring, credit engine, billing). Team autonomy (IoT, finance, partner mgmt). Fault isolation (billing outage doesn't stop capture).
EKS (K8s) over ECS		Portability (multi-cloud for EU data sovereignty), HPA (custom metrics), larger community (CNCF, Helm, observability tools).
PostgreSQL TimescaleDB	+	Single DB for transactional (partners, loans) + time-series (IoT telemetry). Reduces complexity. TimescaleDB compression (90% space savings).
MQTT for IoT		Lightweight (small footprint for constrained devices), offline support (persistent sessions, QoS), publish-subscribe (decouples sensors from consumers).
Blockchain (Hyperledger) for credits		Immutable audit trail (regulatory), buyer trust (verifiable provenance), smart contracts (automated distribution, no reconciliation).
Multi-region deployment		Low-latency for US, EU, India (data residency), DR (region outage protection), compliance (GDPR, data localization).
React Native for field app		Single codebase (iOS/Android) → faster development, consistent features, shared logic (TypeScript). Offline support (WatermelonDB).
Go for logistics		Performance (compiled, no VM), concurrency (goroutines for routing), compiled binary (easy edge deployment).
Hyperledger Fabric for blockchain		Permissioned (approved validators only), high throughput (1000+ TPS vs. Ethereum 15 TPS), privacy (channels for confidential data).

13.2 Projected Capacity by FY30F

- Industrial partners: 1,200+ (2.7x growth from 450)
- Capture capacity: 50M tonnes/year (5.6x growth from 9M)
- IoT sensors deployed: 100,000+ (100x growth from 1,000)

- Daily IoT data points: 100M (100x growth from 1M)
- Daily API requests: 10M (20x growth from 500K)
- Daily Kafka messages: 200M (40x growth from 5M)
- Data warehouse size: 100 TB (10x growth from 10 TB)
- Project financing AUM: \$4.5B (5.3x growth from \$850M)
- Annual revenue: \$1.28B+ (4.6x growth from \$280M)
- Concurrent WebSocket connections: 10,000 (10x growth from 1,000)

13.3 Architecture Readiness Assessment

The architecture is projected to handle these targets with:

- **Uptime:** 99.95% (4.38 hours downtime/year), with 99.99% target for credit issuance (52 minutes/year)
- **P95 latency:** < 1 sec for critical APIs (capture ingestion, credit issuance, partner portal)
- **RTO:** 2 hours (DR), with 1 hour RTO for credit issuance
- **RPO:** 15 minutes (5 min for critical data: credit book, partner profiles, credit issuance records)
- **Scalability:** Horizontal scaling verified by load tests (10x current load, 100x for IoT)

References

1. AWS Well-Architected Framework. (2025). Reliability, Security, Cost Optimization, Performance Efficiency, Operational Excellence Pillars. Amazon Web Services.
2. Verra. (2025). Carbon Capture & Storage Methodology (VM0047), Version 2.0. Verified Carbon Standard.
3. Gold Standard. (2025). Carbon Capture and Utilization Methodology: Requirements for Certification. Gold Standard Foundation.
4. European Commission. (2025). EU ETS Monitoring and Reporting Regulation (MRR), Implementing Regulation (EU) 2025/1234.
5. US Department of Energy. (2025). 45Q Tax Credit: Verification and Attestation Requirements. Internal Revenue Service, Notice 2025-12.

6. Reserve Bank of India. (2025). Master Direction - Non-Banking Financial Companies (NBFC) - Scale Based Regulation (SBR). RBI/2025-26/45.
7. Ministry of Electronics & IT. (2023). Digital Personal Data Protection (DPDP) Act, 2023. Government of India.
8. Martin Fowler. (2018). Microservices: A Definition of This New Architectural Term. martinowler.com.
9. Richardson, C. (2021). Microservices Patterns: With examples in Java. Manning Publications.
10. Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.
11. Google SRE Team. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.
12. Hyperledger Foundation. (2025). Hyperledger Fabric for Carbon Markets: Reference Architecture. Version 2.5.
13. CNCF. (2025). Cloud Native Landscape. Cloud Native Computing Foundation. landscape.cncf.io.
14. Carbon Clean. (2026). Internal Architecture Review: FY25-Q4. Engineering Document (Internal).
15. AWS IoT Core. (2025). Best Practices for Large-Scale Device Deployments (100,000+ devices). Amazon Web Solutions.